

```

        # Simulate O moving on this space:
    ❸ self._spaces = copy.copy(originalSpaces)
        if self._spaces[space] == BLANK:
            self._spaces[space] = O
        if self.isWinner(O):
            oCanWin = True
    if xCanWin:
        boardStr += '\nX can win in one more move.'
    if oCanWin:
        boardStr += '\nO can win in one more move.'
    self._spaces = originalSpaces
    return boardStr

```

First, `super().getBoardStr()` ❶ runs the code inside the parent `TTTBoard` class's `getBoardStr()`, which returns a string of the tic-tac-toe board. We save this string in a variable named `boardStr` for now. With the board string created by reusing `TTTBoard` class's `getBoardStr()`, the rest of the code in this method handles generating the hint. The `getBoardStr()` method then sets `xCanWin` and `oCanWin` variables to `False`, and backs up the `self._spaces` dictionary to an `originalSpaces` variable ❷. Then a `for` loop loops over all board spaces from '1' to '9'. Inside the loop, the `self._spaces` attribute is set to a copy of the `originalSpaces` dictionary, and if the current space being looped on is blank, an X is placed there. This simulates X moving on this blank space for its next move. A call to `self.isWinner()` will determine if this would be a winning move, and if so, `xCanWin` is set to `True`. Then these steps are repeated for O to see whether O could win by moving on this space ❸. This method uses the `copy` module to make a copy of the dictionary in `self._spaces`, so add the following line to the top of *tictactoe.py*:

```
import copy
```

Next, change the line in `main()` so it instantiates a `HintBoard` object instead of a `TTTBoard` object:

```
gameBoard = HintBoard() # Create a TTT board object.
```

Other than this one line change to `main()`, the rest of the program works exactly as before. When you run the program now, the output will look something like this: